

TEXT FILES VS BINARY FILES IN C/CPP PROGRAMMING LANGUAGE

*The major difference between these two is that a **text file** contains textual information in the form of alphabets, digits and special characters or symbols. On the other hand, a **binary file** contains bytes or a compiled version of a text file.*

A FEW OTHER DIFFERENCES ARE LISTED BELOW:

- A **text file** stores data in the form of alphabets, digits and other special symbols by storing their ASCII values and are in a human readable format. For example, any file with a .txt, .c, etc extension. Whereas, a **binary file** contains a sequence or a collection of bytes which are not in a human readable format. For example, files with .exe, .mp3, etc extension. It represents custom data.
- A small error in a **textual file** can be recognized and eliminated when seen. Whereas, a small error in a **binary file** corrupts the file and is not easy to detect.
- Since the data is not human readable it also adds to the security of the content as one might not be able to get data if the structure is not known.
- Now, when it comes to programming there are three major differences between the two, i.e., **Handling of newlines, storage of numbers and representation of EOF (End of File)**. Let's look into these differences in detail:

HANDLING OF NEWLINES

Newline is the *end of the line* or *line ending* or *line break*. It is usually a special character which signifies the end of the line. A **newline** character in a text file is first converted into a carriage return-linefeed combination and then written to the disk. Similarly, when read by a **text file** the carriage return-linefeed combination is converted into a **newline**. However, in a **binary file**, no such conversions take place.

STORAGE OF NUMBERS

In the **text mode**, the function `>> operator` is used to store numerical data in the disk. The texts and the characters are stored one character per byte as it should be (char occupies 1 byte in the memory) and as expected the integers should occupy 4 bytes (depends on the compiler) per number. But this is not the case. For example, we have a number 567392. According to integer storage convention, it should occupy 4 bytes in the disk, but it does not. It occupies 6 bytes, i.e., 1 byte for every digit in the number. Also, the number 56.9057 will occupy 7 bytes in the disk. Thus, we see that each digit in the file is treated as a character in itself and occupies more space than necessary. So, if we have a lot of numerical data then using a text file will not be very memory efficient (but still the syntax used depends on our usage i.e. if we have uses in which a human has to read the file then we can never choose the binary type).

This problem can be solved by using **binary files**. We should open the file in binary mode (using `"ios::binary"` for write and read mode respectively). Then, using the function `read()` or `write()` we can easily store the data in the binary form which shall use only 4 bytes for storing the integer data.

REPRESENTATION OF EOF

Another way the **text mode** and the **binary mode** can be distinguished is on the basis of the representation of the **end-of-file(EOF)**. In the **text mode**, a special character with the ASCII code 26 is inserted at the end of the file. This character when encountered returns the EOF signal to the program.

This is not the case in **binary mode**. In the **binary mode**, we do not have any special character to signify the EOF. It keeps track with the help of the number of characters present in the directory entry of the file.

See Also: <https://www.nayuki.io/page/what-are-binary-and-text-files>